Querying RMBS trustee data in natural language

Utkarsh Malhotra · November 2026

The problem

Monthly trustee data for about 300 Australian RMBS deals sits in Oracle. Each deal holds about 2,000 loans going back 5 years, so the table runs to about 36 million loan-month records. Each record has arrears status, prepayments, LVR, interest rate, balance and a suburb that links to local house prices. The goal is to let people ask questions of this data in ordinary language and get answers they can rely on.

Volume isn't the difficult part. A sensibly partitioned table of 36 million rows answers aggregate queries in seconds, and the prototype below confirms it. The difficult part is definitions. "What are arrears on the 2021 vintage in Western Sydney?" could mean 30 or 90 days past due, by count or by balance, at the latest month or averaged over the year, with hardship loans in or out. A model writing SQL straight against trustee tables will pick one of those readings without saying so, and return a number that looks authoritative. On a credit desk that is worse than no answer at all.

The approach

The language model translates; it never calculates.

It converts the question into a small structured query spec: which metric, grouped by what, filtered how, over which months. A compiler turns the spec into SQL using metric definitions held once in code, and the database does the arithmetic. The model then writes a short answer from the result table, and a check confirms that every number in that answer appears in the table. If a question is ambiguous, the system asks what was meant. If the data can't answer it, the system says so.

Each answer comes with the spec, the Oracle SQL and the result table, so an analyst can verify it in about 10 seconds.

The design at production scale

1. Clean the data before any model sees it. Trustee reports don't use consistent conventions.

- One trustee reports arrears in days, another in missed payments.
- Hardship loans may or may not be counted in arrears.
- A clean-up call can be reported as an ordinary repayment.
- Corrected months are resubmitted on top of the originals.

The first layer is therefore a single canonical loan-month table, with a stable loan key and each exit reason recorded separately: prepayment, repurchase, default and call. Indexed LVR is computed once, as current balance over original valuation scaled by the suburb house price index, and stored as a column. Without this layer, a clean-up call will eventually show up as a wave of prepayments and a CPR spike that never happened.

2. Define each metric once. Every metric lives in code with a single definition. For example:

- 90+ arrears by balance, excluding hardship;
- CPR from voluntary prepayments only, compounded over the period;
- balance-weighted indexed LVR.

The dimensions (deal, originator, vintage, state, suburb, LVR band, rate type, interest only) are listed with their allowed values. The model can only choose from these lists, so it can't make up a metric.

3. The model writes a spec, not SQL. The spec has a fixed shape: 1 to 3 metrics, up to 2 groupings, filters, a date range and an optional ranking. Filling in a form like this is far more reliable than writing correct window functions over a loan panel that shrinks as loans repay. The spec is validated before anything runs.

4. Match names, then compile.

- People write deal and place names loosely: "Harbour 21-1", "blacktown", "HARB21-1". Exact matches and known aliases resolve directly, close misspellings resolve by score, and a name that fits several values comes back as a question.
- The compiler then produces 1 SQL statement for DuckDB, and renders the same statement for Oracle using only constructs Oracle supports.

5. Write the answer from the result, then check it. The answer text is written from the result table only. Every number in the text is checked against the table, and if any fails, the user gets a plain summary of the table instead.

6. Test it on a fixed question set. A set of real questions with hand-checked answers runs after every change to the prompts, the model or a metric definition. It includes ambiguous questions, to confirm the system asks rather than guesses. Without this, the most you can say is that the system sounds right.

At 36 million rows, most questions resolve to deal-month or monthly aggregates, so a small precomputed table answers them, with loan-level detail still available for drill-downs. In production this would run against a read-only replica, with query timeouts and per-deal permissions for anything under NDA.

What the prototype shows

The prototype runs the full design on synthetic data built with the same problems: 50 deals, 21,945 loans and 773,336 loan-months from 2021-10 to 2026-09, in 2 different trustee formats. Rates follow the RBA path and house prices follow suburb indices. 90+ arrears rise from 0.18% to a peak of 1.54% in 2024-08, then fall back to 0.47%.

The cleaning is tested row by row. The raw trustee files hold 774,057 rows, including 721 resubmitted duplicates, and one format reports its 7 clean-up calls as ordinary repayments. After cleaning, the canonical table matches the simulated truth on all 773,336 loan-months, with 0 differences.

The compiler is tested against hand-written SQL. 49 of 49 hand-written queries match the compiler's output, with a worst difference of 1.6e-15. Every compiled query renders to Oracle using only supported constructs.

Volume isn't a problem. With the data copied 36 times, to 27,840,096 loan-months, each of the 34 test questions with a numeric answer runs in under 0.75 seconds on 1 machine with DuckDB's default threads. The slowest single-threaded run takes 3.34 seconds. These are DuckDB timings. Oracle performance depends on partitioning and indexes, which I didn't test.

Compared with direct SQL. I ran the same 40 questions through a second setup: a model writing DuckDB SQL directly against the same clean tables, with the same metric definitions written into its prompt. The result was closer than the argument above implies.

On the 34 questions with a numeric answer, direct SQL got 33 right and the compiled approach 31. Neither returned an incorrect number (0 for direct SQL, 0 for the compiled approach). Every miss on both sides was a refusal to answer.

- The compiled approach didn't recognise a state written out in full ("Queensland", stored as QLD).
- On the 2 questions phrased in missed payments, it asked which day threshold was meant instead of converting payments to days.
- On the 4 ambiguous questions, it asked a clarifying question each time. Direct SQL declined all 4 without explanation, which the scoring counts as wrong.

Overall, the compiled approach got 37 of 40 (92.5%, 95% interval 80.1% to 97.4%) and direct SQL 35 of 40 (87.5%, interval 73.9% to 94.5%). The intervals overlap almost entirely.

My reading: on clean data, the metric definitions do most of the work, whether a compiler applies them or a prompt carries them. Direct SQL only did this well because its prompt contained the full set of definitions written out in prose, and that copy has to be kept in line with the real definitions by hand. What the compiled approach adds is what this test doesn't score: 1 definition per metric used everywhere, the spec and SQL behind every answer, a check on every number in the text, and a question back to the user rather than a silent refusal. I also didn't test direct SQL against the raw trustee tables, which is where the traps below would do damage.

category	questions	compiled approach	direct SQL
simple	10	9 (90%)	10 (100%)
grouped	10	10 (100%)	10 (100%)
time	6	6 (100%)	6 (100%)
trap	8	6 (75%)	7 (88%)
ambiguous	4	4 (100%)	0 (0%)
unsupported	2	2 (100%)	2 (100%)
total	40	37 (92%)	35 (88%)

The traps in the test set

The 8 trap questions target errors that produce valid SQL and wrong credit numbers. On this data:

- counting clean-up calls as prepayments takes CST 2019-1's 2025 CPR from 21.6% to 100.0%;
- counting the exit month as a live loan adds 597 loans that no longer exist to the November 2025 pool;
- a simple average instead of a balance-weighted one moves pool LVR from 51.96% to 50.92%.

With the definitions in its prompt, direct SQL avoided these too (7 of 8, against 6 for the compiled approach). So these questions show what goes wrong without the definitions, rather than what the compiled approach adds on top of them.

I also considered averaging monthly CPR instead of compounding it, but dropped it. At the pool's prepayment rate of 18.1%, the 2 methods differ by only 1.2e-05, which is too small to tell a careful answer from a careless one.

Next steps and limitations

The next tools to build go beyond aggregation:

- roll rate matrices;
- prepayment curves by seasoning;
- cohort loss curves;
- arrears sensitivity to house price falls.

The last needs care. Between 2022 and 2024, rate rises pushed arrears up and house prices down in the same suburbs at the same time, so a simple regression would credit house prices with what was really a rate effect. That tool should control for payment shock and be explicit about what it can't separate.

The test set should also grow from real use, with every question logged and every failure added. The first fix it points to is accepting state names written in full.

Limitations:

- the data is synthetic;
- the test set is 40 questions run once, so each category carries a wide interval. Current Claude models don't accept a sampling temperature, so a rerun could give different answers, and I haven't measured how much they vary;
- the Oracle SQL is generated and syntax-checked, but hasn't been run on Oracle.

All figures are generated from CSV files committed in the repo. The data is synthetic.